

bl-problems

2024年10月25日 21:15

1.rand

rand()产生随机数

rand()%n能产生0-n-1的随机数

- srand() 用于初始化随机数生成器
- 它接受一个 unsigned int 类型的参数作为种子值
- 同样的种子值会产生同样的随机数序列
- srand(time(0)) 使用当前时间戳作为种子
- 每次程序运行时，因为时间不同，所以种子也不同
- 不同的种子会导致 rand() 产生不同的随机数序列

一些注意事项：

- srand 只需要在程序开始时调用一次
- 不要在循环中重复调用 srand
- 如果需要可重现的随机数序列（比如在测试中），可以使用固定的种子值

2.floating number comparison

```
double a = 0.1 + 0.2;
if (a == 0.3) { // 这个比较可能不会为真!
    cout << "相等";
} else {
    cout << "不相等"; // 很可能输出这个
}
```

```
#include <cmath>
const double EPSILON = 0.000001; // 定义一个很小的误差范围

// 比较两个浮点数是否相等
bool isEqual(double a, double b) {
    return fabs(a - b) < EPSILON;
}

// 实际使用
double a = 0.1 + 0.2;
if (isEqual(a, 0.3)) {
    cout << "这样比较更准确";
}
```

- 精度损失：浮点数在计算机中是以二进制分数表示的，某些十进制小数无法精确表示
- 舍入误差：运算过程中的舍入可能导致误差累积
- 不同的计算路径可能得到略微不同的结果

```
// 将所有数值乘以10来避免浮点数
int main() {
    // 4.5 变成 45, 3.0 变成 30, 36.0 变成 360
    for(int pigs = 0; pigs <= 8; pigs++) {
        for(int sheep = 0; sheep <= 12; sheep++) {
            if(pigs * 45 + sheep * 30 == 360) {
                cout << pigs << " * 4.5 + "
                    << sheep << " * 3 = 36" << endl;
            }
        }
    }
    return 0;
}
```